

instalar

INSTALAR E CONFIGURAR O NODEJS

1. INDEX**

1. [Objetivo deste documento.](#)
2. [Pre-requisitos para compreensão deste documento.](#)
3. [Os benefícios que este documento pode me trazer.](#)
4. [Como instalar nodejs no Linux](#)
 1. [Instalar usando a versão da distribuição linux debian ou derivados.](#)
 2. [Instalar a versão de distribuição do site github.](#)
5. [Instalar extensões vscode para nodejs](#)
6. [Referências](#)
7. [Histórico](#)

2. CONTEÚDO

1. Objetivos.
 1. Ensinar como instalar nodejs usando o pacote da distribuição debian;
 2. Ensinar como instalar nodejs usando o pacote da distribuição de github;
 3. Relacionar as extensões do vscode necessárias para trabalhar com nodejs;
 4. 
2. Pre-requisitos:
 1. Sistema operacional debian ou seus derivados;
 2. IDE vscode na microsoft;
 3. Familiaridade com javascript para testar os exemplos de conexão com banco de dados;
 4. 
3. Benefícios deste documento:
 1. Permitir criação de aplicativo back-end para construção de sites.
 2. 
 4. 
5. **Instalar pacote node js no Linux.**
 1. [Dependências do node js](#)
 2. **[*Instalar usando a versão da distribuição linux debian ou derivados.*]**
 1. [Biblioteca c libuv1](#)

```
sudo apt install libuv1
```

2. Instalar o nodejs:

```
sudo apt install nodejs  
# obs: O pacote nodejs contém os binários node e npm.
```

3. Instalar a documentação:

```
sudo apt install nodejs-doc
```

4. Instalar o pacote **npm**. Nota: **npm** são duas coisas: em primeiro lugar, é um [repositório online](#) para a publicação de projetos Node.js de código aberto; em segundo lugar, é um utilitário de linha de comando para interagir com o referido repositório que auxilia na instalação do pacote, gerenciamento de versão e gerenciamento de dependência.

1. Código shellscript

```
sudo apt install npm
```

2. [BACK]

3. Como instalar pacotes via npm:

1. Instalar pacotes no projeto atual. Dica...: [link para config npm](#)

```
npm install package
```

2. Como instalar pacotes npm globalmente:

```
npm install -g package
```

3. Notas:

1. Por padrão, o comando `npm install package` simplesmente instala um pacote em `node_modules`. Ao tentar instalar dependências para seu aplicativo / módulo, você precisa primeiro instalá-las e, em seguida, adicioná-las à seção `dependencies` de seu `package.json`.*.
2. `--save-dev` adiciona o pacote de terceiros a seção `devDependencies` (dependências de desenvolvimento) do pacote. Ele não será instalado quando alguém executar `npm install package` diretamente para instalar seu pacote. Normalmente, ele só é instalado se alguém clonar seu repositório de origem primeiro e depois executá `npm install`-lo.
3. `--save` adiciona o pacote a seção `dependencies` (dependências de terceiros) do pacote. Ele será instalado junto com o pacote sempre que alguém o executar `npm install package`.
4. *Dependências de desenvolvimento* são aquelas dependências necessárias apenas para o desenvolvimento do pacote. Isso pode incluir executores de teste, compiladores, empacotadores, etc. Ambos os tipos de dependências são armazenados no `package.js` no arquivo de pacote. `--save` adiciona a `dependencies`, `--save-dev` adiciona a `devDependencies`.
5. A documentação de instalação do npm pode ser [consultada aqui](#).
6. **ATENÇÃO:** O flag `--save` agora é a opção padrão, desde o NPM 5. Portanto, não é mais explicitamente necessário. É possível executar `npm install package` sem o `--save` para obter o mesmo resultado.

4. [Instalar eslint](#) (ESLint analisa estaticamente seu código para encontrar problemas rapidamente)

```
npm install eslint
```

5. [Instalar nodemon](#) (Os projetos executados com `nodemon` é recarregado toda vez que é modificado em disco e só deve ser usado caso não se use o vscode com opção de debug).

```
sudo npm install -g nodemon
```

6. [Instalar sequelize](#) (Sequelize é um Node.js ORM baseado em promessa para Postgre , MySQL , MariaDB , SQLite e Microsoft SQL Server . Ele

oferece suporte a transações sólidas, relações, carregamento rápido e lento, replicação de leitura e muito mais)

1. [Documentação da classe sequelize](#).
2. Criar pasta do projeto de banco de dados nodejs (Obs: Não pode ser dentro de um link do linux)

```
mkdir NodeSequelize  
cd NodeSequelize
```

3. Iniciar projeto nodejs na pasta NodeSequelize

```
npm init  
# Obs: Se neste momento não solicitar os  
parâmetros básicos do pacote package.json algo está  
errado.
```

4. Instalar pacote principal *sequelize* dentro da pasta *NodeSequelize*:

```
sudo npm install sequelize
```

5. Na pasta *NodeSequelize*, instalar pacote que permite *sequelize* reconheça o [banco de dados mariadb](#):

```
sudo npm install mariadb  
# Obs: se quiser instalar globalmente usar o  
paramento -g
```

6. Opcionalmente pode-se instalar pacote [Sequelize CLI \(Command Line Interface\)](#) caso queira acessar o banco de dados pelo terminal do linux:

```
npm install --save-dev sequelize-cli
```

1. Notas:

1. O flag `--save-dev` registra em *package.json* as dependência no desenvolvimento (*devDependencies*).
 2. O comando `sequelize-cli` é usado para manipular o banco de dados a partir do prompt do linux, porém não vejo necessidade de instalar porque uso o [mysql workbench](#) faz essa tarefa com mais facilidade.
7. [Teste conexão mariadb passando um string](#)

```

/**
 * Conectando-se a um banco de dados.
 * O exemplo abaixo é a forma mais simples para conectar
 um
 * banco de dados com o módulo Sequelize.
 */
// Importando a class Sequelize
const { Sequelize } = require('sequelize');

// Criar o objeto sequelize passando como parâmetro
string de conexão.
const sequelize = new
Sequelize('mariadb://root:password@192.168.15.3:3306/my

/**
 * Testando a conexão
 * Você pode usar a .authenticate() função para testa
 conexão está OK:
 */
sequelize.authenticate().then(
    //Essa função será executada se o banco de dados
conectado
    function () {
        console.log('A conexão foi estabelecida com
sucesso.');
```

8. Teste conexão mariadb passando um objeto json

```

/**
 * Exemplo de como conectar-se ao banco de dados mari
```

```

*
* 0 construtor Sequelize aceita várias opções de con
* Eles estão documentados na [Referência da API]
(https://sequelize.org/master/class/lib/sequelize.js~Se
constructor-constructor) .
*
*/
const
  Sequelize = require('sequelize');

/**
* Cria objeto connection para conecta-ser ao mariadb
* https://sequelize.org/v5/manual/dialects.html
* A seguir criamos uma nova instância do Sequelize u
que
* possui a seguinte sintaxe:
*   conexao = new Sequelize(database, [username=nul
[options={}])
*   onde :
*       1. database: é o nome do schema do ba
acessar.
*       No nosso exemplo usamos
criado no MySQL;
*       2. username: é o nome do usuário de a
exemplo estou
*       usando o usuário root;
*       3. password: é senha do usuário root.
usando a
*       senha do usuário root;
*       4. [options={}] = {host: '192.168.15.
'3306', dialect: 'mariadb'}
* */
const
  connection = new Sequelize('mysql', 'root', 'pass
    { host: '192.168.15.3'
    dialect: 'mariadb' });

/**
* TESTE DA CONEXÃO
* Conecta-se ao banco de dados mariadb com autentica
*
*/
var conexao = connection.authenticate().then(
  function () { console.log('A conexão foi estab
sucesso.')} );
).catch(
  function () { console.error('Não foi possível
dados::', error); }
);

/**
* Fechando a conexão
*
* Sequelize manterá a conexão aberta por padrão e us
* conexão para todas as consultas. Se você precisar

```

```
conexão para todas as consultas. Se você precisar  
* ligue sequelize.close() (Obs: O método sequelize.cl  
retorna uma Promessa).  
*  
*/  
  
//sequelize.close();
```

9.  TOP

7. Instalar a versão de distribuição do site github:

1. Instruções de instalação no ubuntu

1. Registra o repositório com a versão 15.x

```
curl -sL  
https://deb.nodesource.com/setup_15.x | sudo -E  
bash -
```

2. Atualizar lista do repositório

```
sudo apt update
```

3. Atualizar todos os programas da distribuição

```
sudo apt upgrade
```

4. Instalar nodejs

```
sudo apt-get install -y nodejs
```

5. Instalar a documentação:

```
apt install nodejs-doc
```

6. Instalar o gerenciador de pacotes:

```
sudo apt install npm
```

7. [Instalar eslint](#)

```
npm install eslint
```

8. [Acesso a dados com pacote sequelize.org](#)

```
npm install --save sequelize
```

9. [Acesso a dados com pacote sequelize.org](#)

```
npm install --save mariadb
```

10. [Express Framework web rápido, flexível e minimalista para Node.js](#)

```
npm install express --save
```

11. Framework LoopBack:

1. [LoopBack 4 CLI](#)
2. [LoopBack](#) é uma estrutura Node.js e TypeScript premiada , altamente extensível e de código aberto baseada no Express. Ele permite que você crie rapidamente APIs e micros serviços compostos de sistemas de back-end, como bancos de dados e serviços SOAP ou REST.
3. [Getting-started.html](#)

2. [

3. **Instalar extensões vscode para nodejs.**

1. **[AutoFileName:](#)**

1. Ajuda a encontrar o nome completo do arquivo automaticamente.

2. **[Node.js Modules Intellisense:](#)**

1. Autocompletes módulos Node.js em declarações de importação.
2. Implementa autocomplete em módulos JavaScript / TypeScript em instruções de importação.

3. **Live Serve**

1. Servidor web para teste de páginas html.

4. **Beautify json**

1. Enfeitar JSON.

5. **Fix JSON**

1. Corrija o conteúdo JSON usando [jsonic](#)

6. **Node.js Extension Pack**

1. [vscode-eslint](#)

1. A extensão usa a biblioteca ESLint instalada na pasta da área de trabalho aberta. Se a pasta não fornecer um, a extensão procurará uma versão de instalação global.

2. [npm support for VS Code](#).

1. Esta extensão oferece suporte à execução de scripts npm definidos no package.json arquivo e à validação dos módulos instalados em relação às dependências definidas no package.json.

3. [JavaScript \(ES6\) code snippets](#)

1. Esta extensão contém trechos de código para JavaScript na sintaxe ES6 para o editor de código Vs (suporta JavaScript e TypeScript).

4. [Search node_modules](#).

1. Navegue rapidamente e abra arquivos node_modules percorrendo a árvore de pastas.

5. [npm Intellisense](#):

1. Plug-in de código do Visual Studio que autocompleta módulos npm em instruções de importação.

6. [Path Intellisense](#)

1. Plug-in de código do Visual Studio que preenche automaticamente nomes de arquivos.

7. Programa para reiniciar servidor nodejs se o código fonte for trocado

1. <https://nodemon.io/>
2. [Vídeo Aula sobre nodemon](#)

8. [Sequelize VSCode Association Snippet](#)

9. A extensão [Paste JSON as Code](#) Copy JSON, paste as Go, TypeScript, C#, C++ and more.

10. Delphi Keymap

7. 

4. REFERÊNCIAS

1. [Dependências do node js](#)
2. [Biblioteca c libuv1](#)
3. [NodeSource Node.js Binary Distributions](#)
4. [Instruções de instalação no ubuntu](#)
5. [npm Intellisense](#)
6. [AutoFileName](#)
7. [Node.js Modules Intellisense](#)
8. [Instalar eslint](#)
9. [Instalar extensão vscode-eslint](#)
10. [Lista de framework baseado em express](#)
11. [nodejs-install-setup.html](#)
12. [Acesse a banco de dados sequelize](#)
 1. [Acesso a dados com pacote sequelize.org](#)
13. [node_sequelize1](#)
14. [Framework loopback](#)
 1. [Extending-LoopBack-4](#)
15. [get-started-with-creating-a-restful-api-endpoints-in-nodejs-and-expressjs](#)
16. 

5. HISTÓRICO

1. 10/05/2023
 - ☒ Trocado todo o código `<pre></pre>` por ````sh ```` e usado o tema da pasta /css;
2. 04/02/2021
 - ☒ Criado este documento;
3. 06/12/2020
 - ☒ instalar nodejs
 - ☒ [Nodejs Docs](#)
 - ☒ instalar extensões do vscode para nodejs
 - ☒ npm Intellisense
 - ☒ AutoFileName
 - ☒ Node.js Modules Intellisense
 - ☒ eslint nodejs para uso na linha de comandos
 - ☒ Instalar extensão [vscode-eslint](#)

4. 08/02/2021

- ✓ Extensão sequelize para acessar banco de dados:
 - ✓ Instalar sequelize;
 - ✓ Testar acesso ao banco de dados mariadb.

5. 11/02/2021.

- ✓ Documentar a segunda forma de conectar-se ao banco de dados mariadb.
- ✓ Criar um index no arquivo instalar.md.

